DSA Experiments – Code Reference

A easy-to-read guide to all 9 Python programs. Each one has a short **what it does**, **how it works**, **time complexity** and the full code. Every program is menu-driven (type a number to choose an option).

Contents

1. Array Operations
2. Stack Using Array
3. Queue Using Array
4. Singly Linked List
5. Doubly Linked List
6. Insertion Sort
7. Binary Search Tree (BST)
8. Breadth First Search (BFS)
9. Infix to Prefix Conversion

---

1. Array Operations

What it does: Lets you build an array and then insert, delete, find the smallest/largest, or display it.

How it works

- Positions are 1-based for the user (`pos - 1` converts to Python's 0-based index).
- `insert` accepts positions 1 to `len+1` ; `delete` accepts 1 to `len` ; otherwise prints "Invalid position".
- Smallest/largest use the built-in `min()` / `max()` after checking the array isn't empty.

Complexity: insert/delete O(n) (elements shift), min/max O(n).

```
def insert(arr):
    pos = int(input("Enter position: "))
    value = int(input("Enter number to insert: "))

    if 1 <= pos <= len(arr) + 1:
        arr.insert(pos - 1, value)
        print("Array:", arr)
    else:
```

```
        print("Invalid position")


def delete(arr):
    pos = int(input("Enter position to delete: "))

    if 1 <= pos <= len(arr):
        arr.pop(pos - 1)
        print("Array:", arr)
    else:
        print("Invalid position")


arr = list(map(int, input("Enter array elements: ").split()))

while True:
    print("\n1. Insert")
    print("2. Delete")
    print("3. Find Smallest")
    print("4. Find Largest")
    print("5. Display Array")
    print("6. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        insert(arr)

    elif choice == 2:
        delete(arr)

    elif choice == 3:
        if arr:
            print("Smallest number:", min(arr))
        else:
            print("Array is empty")

    elif choice == 4:
        if arr:
            print("Largest number:", max(arr))
        else:
            print("Array is empty")

    elif choice == 5:
        print("Array:", arr)
```

```
    elif choice == 6:
        break

    else:
        print("Invalid choice")
```

---

2. Stack Using Array

What it does: A stack (LIFO – Last In, First Out) with a maximum size of 5.

How it works

- **Push** adds to the end of the list (`append`), unless the stack is full (`len == MAX`).
- **Pop** removes from the end (`pop()`), unless empty.
- **Peek** shows the top element `stack[-1]` without removing it.

Complexity: push, pop, peek all O(1).

```
MAX = 5
stack = []

while True:
    print("\n1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        if len(stack) == MAX:
            print("Stack is Full")
        else:
            value = int(input("Enter number to push: "))
            stack.append(value)
            print("Stack:", stack)

    elif choice == 2:
        if len(stack) == 0:
            print("Stack is Empty")
        else:
            print("Deleted element:", stack.pop())
            print("Stack:", stack)
```

```
    elif choice == 3:
        if len(stack) == 0:
            print("Stack is Empty")
        else:
            print("Top element:", stack[-1])

    elif choice == 4:
        if len(stack) == 0:
            print("Stack is Empty")
        else:
            print("Stack:", stack)

    elif choice == 5:
        break

    else:
        print("Invalid choice")
```

---

3. Queue Using Array

What it does: A queue (FIFO – First In, First Out) with a maximum size of 5.

How it works

- **Enqueue** adds to the back (`append`), unless full.
- **Dequeue** removes from the front (`pop(0)`), unless empty.
- **Peek** shows the front element `queue[0]`.

Complexity: enqueue O(1), dequeue O(n) because `pop(0)` shifts every element.

```
MAX = 5
queue = []

while True:
    print("\n1. Enqueue")
    print("2. Dequeue")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        if len(queue) == MAX:
            print("Queue is Full")
```

```
        else:
            value = int(input("Enter number to insert: "))
            queue.append(value)
            print("Queue:", queue)

    elif choice == 2:
        if len(queue) == 0:
            print("Queue is Empty")
        else:
            print("Deleted element:", queue.pop(0))
            print("Queue:", queue)

    elif choice == 3:
        if len(queue) == 0:
            print("Queue is Empty")
        else:
            print("Front element:", queue[0])

    elif choice == 4:
        if len(queue) == 0:
            print("Queue is Empty")
        else:
            print("Queue:", queue)

    elif choice == 5:
        break

    else:
        print("Invalid choice")
```

---

4. Singly Linked List

What it does: A chain of nodes where each node stores `data` and a pointer `next` to the following node. `head` points to the first node.

How it works

- **Insert at beginning:** new node points to the old head, then becomes the head.
- **Insert at end:** walk to the last node (`temp.next` is None) and attach the new node.
- **Delete at beginning:** move `head` to `head.next`.
- **Delete at end:** walk until `temp.next.next` is None, then cut the last node off (`temp.next = None`). A single-node list is handled separately.
- **Display:** walk from head, printing each `data`.

Complexity: insert/delete at beginning O(1); at end O(n).

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


head = None


def insert_beginning(value):
    global head
    new_node = Node(value)
    new_node.next = head
    head = new_node


def insert_end(value):
    global head
    new_node = Node(value)

    if head is None:
        head = new_node
        return

    temp = head
    while temp.next:
        temp = temp.next

    temp.next = new_node


def delete_beginning():
    global head
    if head is None:
        print("List is Empty")
    else:
        head = head.next


def delete_end():
    global head

    if head is None:
        print("List is Empty")
```

```
    elif head.next is None:
        head = None
    else:
        temp = head
        while temp.next.next:
            temp = temp.next
        temp.next = None


def display():
    temp = head

    if temp is None:
        print("List is Empty")
        return

    while temp:
        print(temp.data, end=" ")
        temp = temp.next
    print()


while True:
    print("\n1. Insert at Beginning")
    print("2. Insert at End")
    print("3. Delete from Beginning")
    print("4. Delete from End")
    print("5. Display")
    print("6. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter number: "))
        insert_beginning(value)
        display()

    elif choice == 2:
        value = int(input("Enter number: "))
        insert_end(value)
        display()

    elif choice == 3:
        delete_beginning()
        display()
```

```
    elif choice == 4:
        delete_end()
        display()

    elif choice == 5:
        display()

    elif choice == 6:
        break

    else:
        print("Invalid choice")
```

---

5. Doubly Linked List

What it does: Like the singly linked list, but each node also has a `prev` pointer to the node before it, so links go both ways.

How it works

- **Insert at beginning:** new node's `next` = old head, old head's `prev` = new node, then update `head`.
- **Insert at end:** walk to the last node, set `temp.next = new_node` and `new_node.prev = temp`.
- **Delete at beginning:** move `head` forward and clear the new head's `prev`.
- **Delete at end:** walk to the last node and use `temp.prev.next = None` (no need for the `next.next` trick).

Complexity: beginning operations O(1); end operations O(n) (no tail pointer is kept).

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None


head = None


def insert_beginning(value):
    global head
    new_node = Node(value)
    new_node.next = head
```

```
    if head is not None:
        head.prev = new_node

    head = new_node


def insert_end(value):
    global head
    new_node = Node(value)

    if head is None:
        head = new_node
        return

    temp = head
    while temp.next:
        temp = temp.next

    temp.next = new_node
    new_node.prev = temp


def delete_beginning():
    global head

    if head is None:
        print("List is Empty")
        return

    head = head.next

    if head is not None:
        head.prev = None


def delete_end():
    global head

    if head is None:
        print("List is Empty")
        return

    if head.next is None:
        head = None
        return
```

```
    temp = head
    while temp.next:
        temp = temp.next

    temp.prev.next = None


def display():
    temp = head

    if temp is None:
        print("List is Empty")
        return

    while temp:
        print(temp.data, end=" ")
        temp = temp.next
    print()


while True:
    print("\n1. Insert at Beginning")
    print("2. Insert at End")
    print("3. Delete from Beginning")
    print("4. Delete from End")
    print("5. Display")
    print("6. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter number: "))
        insert_beginning(value)
        display()

    elif choice == 2:
        value = int(input("Enter number: "))
        insert_end(value)
        display()

    elif choice == 3:
        delete_beginning()
        display()

    elif choice == 4:
```

```
        delete_end()
        display()

    elif choice == 5:
        display()

    elif choice == 6:
        break

    else:
        print("Invalid choice")
```

---

6. Insertion Sort

What it does: Sorts a list of numbers in ascending order.

How it works

- Treat the left part of the array as already sorted.
- Take the next element (`key`), shift every larger element in the sorted part one step right, then drop `key` into the gap.
- Repeat for every element from index 1 to the end.

Complexity: O(n²) worst/average, O(n) best (already sorted). Stable and in-place.

```
arr = list(map(int, input("Enter array elements: ").split()))

for i in range(1, len(arr)):
    key = arr[i]
    j = i - 1

    while j >= 0 and arr[j] > key:
        arr[j + 1] = arr[j]
        j -= 1

    arr[j + 1] = key

print("Sorted array:", arr)
```

---

7. Binary Search Tree (BST)

What it does: Asks how many nodes you want, reads that many values, builds a Binary Search Tree (smaller values go left, larger go right, duplicates are ignored), then prints the tree sideways and its three traversals.

How it works

- **Insert:** recursively go left or right until an empty spot is found, then create the node there.
- **Display:** visits right subtree, then the node, then left subtree, adding 5 spaces of indentation at each level. This prints the tree rotated 90° – the root is at the left edge and the right children appear on top.
- **Traversals:**
  - Inorder (Left, Root, Right) → prints values in sorted order
  - Preorder (Root, Left, Right)
  - Postorder (Left, Right, Root)

Complexity: insert O(h) where h is the tree height – O(log n) when balanced, O(n) worst case. Display and traversals O(n).

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

def insert(root, value):
    if root is None:
        return Node(value)
    if value < root.data:
        root.left = insert(root.left, value)
    elif value > root.data:
        root.right = insert(root.right, value)
    return root

def display(root, space=0):
    if root is None:
        return
    space += 5
    display(root.right, space)
    print()
    print(" " * space + str(root.data))
    display(root.left, space)
```

```
def inorder(root):
    if root is not None:
        inorder(root.left)
        print(root.data, end=" ")
        inorder(root.right)

def preorder(root):
    if root is not None:
        print(root.data, end=" ")
        preorder(root.left)
        preorder(root.right)

def postorder(root):
    if root is not None:
        postorder(root.left)
        postorder(root.right)
        print(root.data, end=" ")

root = None
n = int(input("How many nodes do you want to insert? "))
for i in range(n):
    value = int(input(f"Enter value for node {i + 1}: "))
    root = insert(root, value)

print("\nConstructed Binary Search Tree:")
display(root)
print("\n\nInorder Traversal:")
inorder(root)
print("\n\nPreorder Traversal:")
preorder(root)
print("\n\nPostorder Traversal:")
postorder(root)
```

---

8. Breadth First Search (BFS)

What it does: Builds an undirected graph from edges you enter, then visits vertices level by level starting from a vertex you choose.

How it works

- The graph is a dictionary (adjacency list): `vertex → list of neighbours`. `add_edge` adds the link in both directions.
- BFS uses a **queue** (`deque`) and a `visited` set. Pop a vertex, print it, then add all unvisited neighbours to the queue.

- Option 3 prints the raw graph dictionary.

Complexity: O(V + E), where V = vertices and E = edges.

```
from collections import deque

graph = {}


def add_edge(u, v):
    if u not in graph:
        graph[u] = []
    if v not in graph:
        graph[v] = []

    graph[u].append(v)
    graph[v].append(u)


def bfs(start):
    visited = set()
    queue = deque([start])
    visited.add(start)

    while queue:
        vertex = queue.popleft()
        print(vertex, end=" ")

        for neighbour in graph.get(vertex, []):
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)

    print()


while True:
    print("\n1. Add Edge")
    print("2. BFS Traversal")
    print("3. Display Graph")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        u = input("Enter first vertex: ")
        v = input("Enter second vertex: ")
```

```
        add_edge(u, v)

    elif choice == 2:
        start = input("Enter starting vertex: ")

        if start in graph:
            print("BFS Traversal:")
            bfs(start)
        else:
            print("Vertex not found")

    elif choice == 3:
        print(graph)

    elif choice == 4:
        break

    else:
        print("Invalid choice")
```

---

9. Infix to Prefix Conversion (Using Stack)

What it does: Converts an expression like `A+B*C` (infix) into prefix form `+A*BC`.

How it works

1. **Reverse** the expression and **swap** every `(` with `)` and vice-versa.
2. Scan left to right:
   - Letter/digit → append to result.
   - `(` → push on stack.
   - `)` → pop to the result until `(` is found, then discard `(`.
   - Operator → pop operators with **higher** precedence from the stack, then push the current operator.
3. Pop whatever is left on the stack into the result.
4. **Reverse** the result to get the prefix expression.

Precedence used: `^` (3) > `* / %` (2) > `+ -` (1).

Example: `A+B*C` → `+A*BC`

Complexity: O(n).

```
def precedence(op):
    if op == '^':
        return 3
    elif op in '*/%':
        return 2
    elif op in '+-':
        return 1
    return 0


def infix_to_prefix(expression):
    expression = expression[::-1]
    expression = ''.join(')' if c == '(' else '(' if c == ')' else c
                         for c in expression)

    stack = []
    result = ""

    for char in expression:
        if char.isalnum():
            result += char

        elif char == '(':
            stack.append(char)

        elif char == ')':
            while stack and stack[-1] != '(':
                result += stack.pop()
            if stack:
                stack.pop()

        else:
            while (stack and stack[-1] != '(' and
                   precedence(stack[-1]) > precedence(char)):
                result += stack.pop()
            stack.append(char)

    while stack:
        result += stack.pop()

    return result[::-1]


expression = input("Enter infix expression: ")
print("Prefix expression:", infix_to_prefix(expression))
```